

Subgraph Isomorphism Problem (NP-Complete Problem)

1. Problem Understanding

Problem Statement

Given two graphs:

- **Pattern graph $G_1 = (V_1, E_1)$** – the smaller graph.
- **Target graph $G_2 = (V_2, E_2)$** – the larger graph.

The objective is to determine whether **G_1 exists as a subgraph of G_2** .

A subgraph isomorphism exists if we can map every vertex of G_1 to a different vertex of G_2 such that every edge in G_1 is also preserved in G_2 .

The mapping must satisfy:

1. Every vertex of G_1 is mapped to exactly one vertex of G_2 .
2. No two vertices of G_1 can be mapped to the same vertex of G_2 .
3. If (u, v) is an edge in G_1 , then $(\text{mapping}[u], \text{mapping}[v])$ must be an edge in G_2 .
4. The mapping must be consistent for all vertices and edges.
5. Extra vertices and extra edges may exist in G_2 .

Therefore, the problem asks:

Does there exist an injective mapping from G_1 to G_2 that preserves all edges of G_1 ?

2. Constraints and Constraint Analysis

The important constraints are:

2.1 Vertex Constraint

Every vertex in the pattern graph must be mapped to a vertex in the target graph.

If:

$$|V_1| > |V_2|$$

then a valid mapping is impossible.

Therefore:

$$|V_1| \leq |V_2|$$

is a necessary condition.

2.2 One-to-One Mapping

A vertex in G_2 cannot be assigned to more than one vertex in G_1 .

For example:

G_1 : A, B, C

cannot be mapped as:

$$A \rightarrow 1$$

$$B \rightarrow 1$$

$$C \rightarrow 2$$

because A and B are mapped to the same target vertex.

2.3 Degree Constraint

A pattern vertex cannot be mapped to a target vertex with a smaller degree.

If:

$$\text{degree}(u) > \text{degree}(v)$$

then u cannot be mapped to v.

Therefore, candidate vertices can be reduced using:

$$\text{degree}(\text{target}) \geq \text{degree}(\text{pattern})$$

This is an important pruning condition.

2.4 Edge Preservation

For every edge:

$$(u, v) \in E_1$$

the corresponding target vertices must satisfy:

$$(\text{mapping}[u], \text{mapping}[v]) \in E_2$$

If this condition fails, the current mapping is immediately rejected.

2.5 Injective Mapping

The mapping must be one-to-one:

$$\text{mapping}[u] \neq \text{mapping}[v]$$

for every two different vertices u and v.

2.6 Priority of Constraints

The constraints can be checked in the following order:

1. Number of vertices
2. Degree feasibility
3. Candidate generation
4. One-to-one assignment
5. Edge preservation
6. Backtracking when a mapping becomes invalid

This order reduces unnecessary searching.

3. Solution Development

Backtracking Approach

The Subgraph Isomorphism problem can be solved using **backtracking**.

The basic idea is:

1. Select one vertex from the pattern graph.
2. Find possible vertices in the target graph.
3. Try one candidate.
4. Check whether the partial mapping is valid.
5. If valid, recursively map the next pattern vertex.
6. If the mapping fails, undo the assignment.
7. Continue until all pattern vertices are mapped.

This systematically explores possible mappings while eliminating invalid mappings as early as possible.

Pruning Techniques

To improve the basic backtracking algorithm, the following pruning techniques are used:

1. Degree Pruning

A pattern vertex u can only be mapped to a target vertex v if:

$$\text{degree}(v) \geq \text{degree}(u)$$

2. Used-Vertex Pruning

A target vertex that has already been assigned cannot be used again.

3. Neighbor Consistency

When assigning a pattern vertex, all already-mapped neighbors must have corresponding edges in the target graph.

4. Most-Constrained Vertex First

The pattern vertex with the highest degree is selected first.

This generally reduces the search space because highly connected vertices have fewer feasible candidates.

4. Algorithm

Step-by-Step Algorithm

Input:

- Pattern graph G1
- Target graph G2

Output:

- True if G1 is isomorphic to a subgraph of G2
- False otherwise

Algorithm

SUBGRAPH_ISOMORPHISM(G1, G2)

1. If number of vertices in G1 > number of vertices in G2:
 return False
2. Calculate the degree of every vertex in G1 and G2.
3. For every vertex u in G1:
 Create a candidate list containing target vertices v
 such that $\text{degree}(v) \geq \text{degree}(u)$.
4. Sort pattern vertices by decreasing degree.
5. Start BACKTRACKING with an empty mapping.
6. In BACKTRACKING:
 - a. If all pattern vertices are mapped:
 return True

- b. Select the next pattern vertex u .
- c. For every candidate target vertex v :
 - i. Skip v if it is already used.
 - ii. Check degree feasibility.
 - iii. Check whether all already-mapped neighbors of u are connected to v in G_2 .
 - iv. If valid:
 - map $u \rightarrow v$
 - recursively continue
 - v. If recursion fails:
 - remove $u \rightarrow v$
- 7. If no candidate works:
 - return False

5. Python Implementation

```
def subgraph_isomorphism(pattern, target):
    """
    Determines whether pattern graph is isomorphic
    to a subgraph of target graph.

    Graph representation:
    {
        vertex: set(neighbors)
    }
    """

    # Number of vertices condition
    if len(pattern) > len(target):
        return False, {}

    # Degree calculation
    pattern_degree = {
        v: len(pattern[v]) for v in pattern
    }

    target_degree = {
        v: len(target[v]) for v in target
```

```

}

# Candidate generation using degree pruning
candidates = {}

for u in pattern:
    candidates[u] = [
        v for v in target
        if target_degree[v] >= pattern_degree[u]
    ]

    # No possible target vertex
    if not candidates[u]:
        return False, {}

# Most-constrained vertices first
pattern_vertices = sorted(
    pattern,
    key=lambda x: (
        len(candidates[x]),
        -pattern_degree[x]
    )
)

mapping = {}
used = set()

def is_valid(u, v):
    # Target vertex must not already be used
    if v in used:
        return False

    # Check all already-mapped neighbors
    for neighbor in pattern[u]:
        if neighbor in mapping:
            mapped_neighbor = mapping[neighbor]

            if mapped_neighbor not in target[v]:
                return False

    return True

def backtrack(index):
    # All pattern vertices successfully mapped
    if index == len(pattern_vertices):
        return True

```

```

    u = pattern_vertices[index]

    for v in candidates[u]:

        if not is_valid(u, v):
            continue

        # Assign mapping
        mapping[u] = v
        used.add(v)

        # Recursive search
        if backtrack(index + 1):
            return True

        # Undo assignment
        used.remove(v)
        del mapping[u]

    return False

result = backtrack(0)

if result:
    return True, mapping
else:
    return False, {}

# -----
# Example
# -----

pattern = {
    1: {2},
    2: {1, 3},
    3: {2}
}

target = {
    'A': {'B', 'D'},
    'B': {'A', 'C'},
    'C': {'B', 'E'},
    'D': {'A'},

```

```

        'E': {'C'}
    }

    result, mapping = subgraph_isomorphism(pattern, target)

    if result:
        print("Subgraph Isomorphism Exists")
        print("Vertex Mapping:", mapping)
    else:
        print("No Subgraph Isomorphism Exists")

```

6. Sample Input and Output

Pattern Graph

1 -- 2 -- 3

The pattern graph contains three vertices and two edges.

Target Graph

A -- B -- C -- E
|
D

A valid mapping is:

1 → A
2 → B
3 → C

The edges:

1 -- 2
2 -- 3

are preserved as:

A -- B
B -- C

Output

Subgraph Isomorphism Exists
Vertex Mapping: {1: 'A', 2: 'B', 3: 'C'}

The exact mapping may differ depending on the order in which candidates are explored.

7. Correctness and Justification

The algorithm is correct because it considers all feasible mappings while rejecting mappings that violate necessary conditions.

Case 1: Invalid Number of Vertices

If:

$$|V_1| > |V_2|$$

there cannot be an injective mapping.

Therefore, returning False is correct.

Case 2: Degree Constraint

Suppose a pattern vertex has degree 3 but the selected target vertex has degree 2.

The pattern vertex requires three neighboring vertices, but the target vertex has only two.

Therefore, the mapping is impossible and can safely be rejected.

Case 3: Used Vertex

If a target vertex has already been assigned to another pattern vertex, assigning it again violates the one-to-one mapping requirement.

Therefore, it is rejected.

Case 4: Edge Preservation

For every already-mapped neighbor u' of the current pattern vertex u , the algorithm checks:

$$(\text{mapping}[u], \text{mapping}[u']) \in E_2$$

If the required edge does not exist, the mapping cannot represent the pattern graph.

Therefore, that candidate is rejected.

Case 5: Complete Mapping

If all vertices of the pattern graph have been mapped and every required edge has been preserved, an injective edge-preserving mapping has been found.

Therefore:

Subgraph Isomorphism Exists

is correct.

Backtracking Completeness

If a candidate mapping fails, the algorithm removes the assignment and tries another candidate.

Thus, every feasible candidate mapping that survives the pruning rules can be explored.

Therefore, if the algorithm returns False, no valid subgraph isomorphism exists.

8. Time and Space Complexity

Let:

n = number of vertices in the pattern graph

m = number of vertices in the target graph

The algorithm may need to try many possible vertex mappings.

In the worst case, approximately:

$$O(m \times (m - 1) \times (m - 2) \times \dots \times (m - n + 1))$$

possible mappings may be explored.

This is bounded by:

$$O(m^n)$$

for a simple upper-bound representation.

Therefore, the worst-case running time is exponential.

The backtracking space required for:

- mapping
- used vertices
- recursion stack
- candidate lists

is approximately:

$$O(nm)$$

excluding the input graph storage.

The degree and neighbor pruning significantly reduce the practical search space, although they do not change the worst-case exponential nature of the problem.

9. Why Subgraph Isomorphism Is NP-Complete

The **decision version** of the Subgraph Isomorphism problem is NP-complete.

9.1 It Belongs to NP

A problem belongs to NP if a proposed solution can be verified in polynomial time.

For Subgraph Isomorphism, suppose someone provides a mapping:

$$f : V_1 \rightarrow V_2$$

We can verify:

1. Every pattern vertex has a target vertex.
2. No two pattern vertices have the same target vertex.
3. Every pattern edge maps to a target edge.

These checks can be performed in polynomial time.

Therefore:

Subgraph Isomorphism $\in$ NP

9.2 It Is NP-Hard

To prove NP-hardness, we can reduce the **Clique problem** to Subgraph Isomorphism.

Suppose we are given a graph G and an integer k .

The Clique problem asks whether G contains a complete subgraph of k vertices.

Construct a pattern graph:

$$G_1 = K_k$$

where K_k is a complete graph containing k vertices.

Then:

G contains a k -clique

if and only if:

K_k is isomorphic to a subgraph of G

Therefore, solving Subgraph Isomorphism would also solve Clique.

Since Clique is NP-complete, Subgraph Isomorphism is NP-hard.

Combining both results:

Subgraph Isomorphism $\in$ NP

and

Subgraph Isomorphism is NP-hard

Therefore:

Subgraph Isomorphism is NP-complete.

10. Application of Concepts

The solution applies several important algorithmic concepts:

Backtracking

Backtracking systematically explores possible vertex mappings and reverses decisions when a mapping becomes invalid.

Constraint Satisfaction

Each pattern vertex must satisfy multiple constraints before being assigned to a target vertex.

Pruning

Degree checking and neighbor consistency eliminate impossible candidates early.

Graph Theory

The algorithm uses:

- Vertices
- Edges
- Degree
- Adjacency
- Injective mappings
- Subgraphs
- Isomorphism

Complexity Analysis

The exponential worst-case complexity explains why an exact solution becomes difficult for large graphs.

NP-Completeness

The Clique reduction establishes the theoretical difficulty of the decision problem.

11. Advantages of the Proposed Solution

1. **Correct:** It checks vertex and edge constraints systematically.
2. **Complete:** It can find a valid mapping if one exists.
3. **Efficient in practice:** Degree and neighbor pruning reduce unnecessary searches.
4. **Easy to understand:** Backtracking provides a clear recursive structure.
5. **No external libraries required:** The Python implementation uses standard language features.
6. **Suitable for small and medium graphs:** It can solve many practical instances efficiently.
7. **Demonstrates NP-complete behavior:** Large worst-case instances can require exponential search.

12. Limitations

- The main limitation is the exponential worst-case running time.
- For large graphs, the number of possible mappings becomes extremely large.
- For example, if the target graph has many vertices, the algorithm may need to examine a very large number of possible assignments before determining whether a valid mapping exists.
- Therefore, the pruning techniques improve practical performance but cannot eliminate the fundamental computational difficulty of the NP-complete problem.

13. Conclusion

The Subgraph Isomorphism problem determines whether a smaller pattern graph can be mapped into a larger target graph while preserving vertices and edges.

A backtracking algorithm with constraint checking and pruning provides an exact and feasible solution. Degree constraints eliminate impossible candidates, used-vertex checking guarantees an injective mapping, and neighbor consistency guarantees edge preservation.

The algorithm is correct because it explores all feasible mappings while safely eliminating mappings that violate necessary constraints.

The decision version belongs to NP because a proposed mapping can be verified in polynomial time. It is NP-hard through a polynomial-time reduction from the Clique problem.

Hence:

Subgraph Isomorphism is an NP-complete problem.

The problem demonstrates why constraint-based backtracking and intelligent pruning are important techniques for solving computationally difficult graph problems.

Q1
Subgraph Isomorphism
Problem (NP-Complete
Problem)
100 marks

1/1 answered

Determine whether a smaller graph exists as a subgraph of a larger graph. Clearly define constraints related to vertex mapping and edge preservation. Analyze how constraints limit feasible mappings. Develop a backtracking-based solution. Apply constraint checking to eliminate invalid matches. Justify correctness and discuss its NP-Complete classification.

Your Code

```

1 def read_graph():
2     n, m = map(int, input().split())
3     adj = [[0] * n for _ in range(n)]
4     for _ in range(m):
5         u, v = map(int, input().split())
6         adj[u][v] = 1
7         adj[v][u] = 1
8     return n, adj
9
10 def is_consistent(mapping, used, h_adj, g_adj, h_n):
11     for hv1 in range(h_n):
12         if mapping[hv1] == -1:
13             continue
14         gv1 = mapping[hv1]
15         for hv2 in range(h_n):
16             if h_adj[hv1][hv2] == 1:
17                 if mapping[hv2] == -1:
18                     continue
19                 gv2 = mapping[hv2]
20                 if g_adj[gv1][gv2] == 0:
21                     return False
22         return True
23
24 def backtrack(pos, mapping, used, h_adj, g_adj, h_n, g_n):
25     if pos == h_n:
26         return True
27     for gv in range(g_n):
28         if not used[gv]:
29             mapping[pos] = gv
30             used[gv] = True
31             if is_consistent(mapping, used, h_adj, g_adj, h_n):
32                 if backtrack(pos + 1, mapping, used, h_adj, g_adj, h_n):
33                     return True
34             mapping[pos] = -1
35             used[gv] = False
36     return False
37
38 def main():
39     g_n, g_adj = read_graph()
40     h_n, h_adj = read_graph()
41     if h_n > g_n:
42         print("NO")
43         return
44     mapping = [-1] * h_n
45     used = [False] * g_n
46     if backtrack(0, mapping, used, h_adj, g_adj, h_n, g_n):
47         print("YES")
48         for i in range(h_n):
49             print(i, "->", mapping[i])
50     else:
51         print("NO")
52
53 if __name__ == "__main__":
54     main()

```

Input

```

4 4
0 1
1 2
2 3

```

Output

```

YES
0 -> 0
1 -> 1
2 -> 2

```

Submitted



Search

